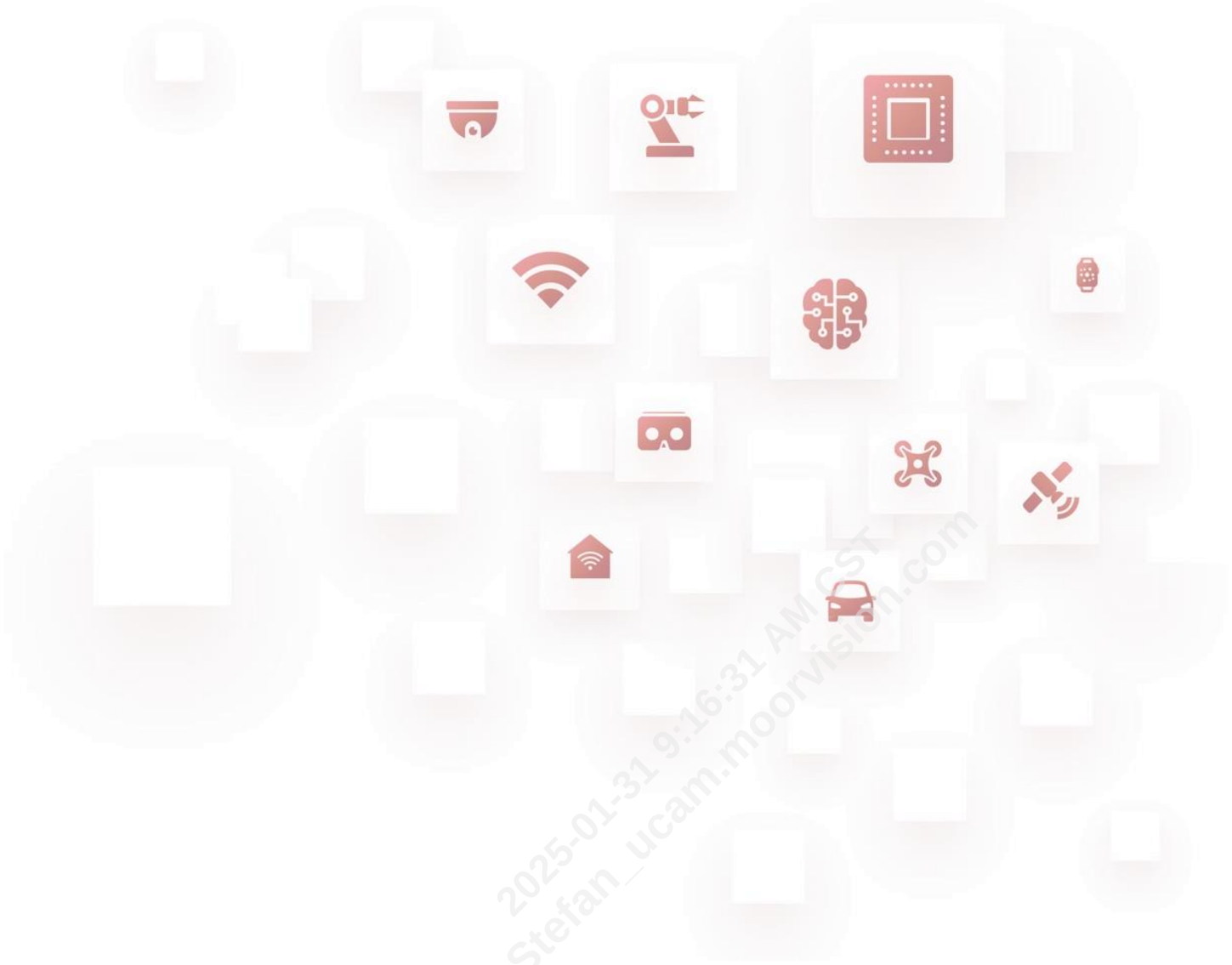
Thundercomm

Empowering Every IoT Device with Our Technology



2025-01-31 9:16:31 AM CST
Stefan_ucam.moorvision.com

Thundercomm TurboX™ C610 Open Kit

Camera Bringup Guide

Rev. V1.0

Jan 20, 2021

DN: TC_C610_6210

Revision History

Revision	Date	Description
1.0	Jan 20, 2020	Initial version

Thundercomm Confidential
2025-01-31 9:16:31 AM CST
Stefan_ucam.moorvision.com

Copyright© 2021 Thundercomm Technology Co., Ltd. All rights reserved.

Table List

Table 2-1: Sensor information nodes– slaveInfo
Table 2-2: Sensor information nodes– refAddrInfo
Table 2-3: Sensor information nodes– resolutionInfo
Table 2-4: Sensor information nodes– exposureControlInfo
Table 2-5: Sensor information nodes– streamOnSettings
Table 2-6: Sensor information nodes– groupHoldOnSettings
Table 2-7: Sensor information nodes– initSettings
Table 2-8: Sensor information nodes– testPatternInfo
Table 2-9: Sensor information nodes– testPatternData
Table 2-10: Sensor information nodes– colorLevelInfo
Table 2-11: Sensor information nodes– Black region
Table 2-12: Sensor information nodes– Pixel array
Table 2-13: Sensor information nodes– Delay
Table 2-14: Sensor information nodes- Sensor property
Table 2-15: CameraPosition
Table 2-16: LensInformation
Table 2-17: LensInformationList
Table 2-18: ModuleConfiguration
Table 2-19: ModuleGroup
Table 2-20: CameraModuleData
Table 2-21: Camera_LDO
Table 2-22: CCI-based sensor driver
Table 2-23: OIS node
Table 2-24: EEPROM node
Table 2-25: EEPROM slaveInfo
Table 2-26: EEPROM memoryMap
Table 2-27: EEPROM formatInfo
Table 2-28: Actuator slave information
Table 2-29: Actuator register information
Table 2-30: Actuator init information
Table 2-31: Actuator tuning information

Thundercomm Confidential
2025-01-31 9:16:31 AM CST
Stefan_ucam.moorvision.com

Copyright© 2021 Thundercomm Technology Co., Ltd. All rights reserved.

Table 2-32: Actuator regulator

Table 2-33: Actuator regulator (cont.)

Table 2-34: SOC-based actuator driver

Table 2-35: CCI-based actuator driver

Table 2-36: Flash driver data

Table 2-37: Flash I2C information

Table 2-38: Flash trigger information

Table 2-39: PMIC-based LED flash driver

Table 2-40: QUP/I2C-based LED flash driver

Table 2-41: CCI-based LED flash driver

Thundercomm Confidential

2025-01-31 9:16:31 AM CST
Stefan_ucam.moorvision.com

Copyright© 2021 Thundercomm Technology Co., Ltd. All rights reserved.

Contents

Chapter 1. Introduction ..... - 2 -

1. Purpose ..... - 2 -

2. Conventions ..... - 2 -

Chapter 2. Bring up Camera Module Driver ..... - 3 -

1. File locations ..... - 3 -

2. Generate binary files ..... - 3 -

3. Bring up camera sensor ..... - 4 -

3.1. Configure sensor software ..... - 4 -

3.2. Sensor hardware configuration ..... - 19 -

3.3. Sensor kernel nodes ..... - 19 -

3.4. Configure CCI operation speed ..... - 24 -

3.5. Use different interfaces ..... - 24 -

3.6. Power regulator configuration ..... - 24 -

3.7. Clock configuration ..... - 25 -

3.8. Power regulator configuration ..... - 26 -

4. EEPROM software configuration ..... - 27 -

4.1. EEPROM-specific XML ..... - 27 -

4.2. EEPROM information node ..... - 27 -

5. Bring up actuator ..... - 33 -

5.1. Actuator software configuration ..... - 33 -

5.2. Actuator hardware configuration ..... - 37 -

6. Flash software configuration ..... - 39 -

Appendix 1. Intellectual Property Statement ..... - 44 -

Appendix 2. Trademarks ..... - 45 -

Thundercomm Confidential
2025-01-31 9:16:31 AM CST
stefan_ucam_mod@visionn.com

Chapter 1. Introduction

1. Purpose

This document provides guidelines for driver development and describes how to bring up the camera module driver on the Thundercomm TurboX C610 LE (Linux Embedded) platform. The document also describes the terms used in the sensor driver and provides examples of the sensor configuration parameters.

2. Conventions

- Function declarations, function names, type declarations, attributes, and code samples appear in a different format, for example, **cp armcc armcpp**.
- Code variables appear in angle brackets, for example, **<number>**.
- Button, tool, and key names appear in bold font, for example, click **Save** or press **Enter**.
- Commands to be entered appear in a different font, for example:

  $ adb devices
- On the host computer use $ as shell prompt, for example:

  $ logcat
- On the target device use # as shell prompt, for example:

  # adb shell

Thundercomm Confidential
2025-01-31 9:16:31 AM CST
Stefan_ucam.moorvision.com

Chapter 2. Bring up Camera Module Driver

Main steps:

- To locate the driver and module configuration XML files, see *File locations*.
- To generate the binary files, see *Generate binary files*.
- Compile the build.

1. File locations

The files at the following locations are modified during build compilation.

- **Sensor driver XML files:**

  apps_proc/src/vendor/qcom/proprietary/chi-cdk/vendor/sensor/default/<sensor_name>/<sensor_name)_sensor.xml

- **Module configuration files:**

  apps_proc/src/vendor/qcom/proprietary/chi-cdk/vendor/module/<module_name>_module.xml

- **Kernel dts files:**

  kernel/msm-4.9/arch/arm64/boot/dts/qcom/<target_name>-camera-sensor-<platform>.dtsi

- **Submodule driver XML files:**

  apps_proc/src/vendor/qcom/proprietary/chi-cdk/vendor/<sub-module_name>/default/<sub-module_name_sub-module>.xml

  Example: *chi-cdk/vendor/actuator/default/<actuator_name>_actuator.xml*

- **The binary files are included in device vendor makefile:**

  vendor/qcom/proprietary/chi-cdk/product.mk

  Example: *PRODUCT_PACKAGES +=com.qti.sensormodule.<sensor_name>.bin*

2. Generate binary files

Prerequisites:

Ensure that all register settings and data sheets for the bringup of sensor, actuator and EEPROM (Electrically Erasable Programmable Read-Only Memory) are ready before driver creation.

Step 1. Add sensor binary file to:

vendor/qcom/proprietary/chi-cdk/cdk/tools/buildbins.yaml

Example:

```
sensor_drivers:
- com.qti.sensormodule.TS_imx577:
      - sensor/default/imx577/imx577_sensor.xml
      - module/altek1_imx577_module.xml
      - eeprom/sunny_gt24c64/gt24c64_eeprom.xml
```

Thundercomm Confidential
2024-03-31 16:31 AM CST
Stefan_icam@horvision.com

Step 2. To generate the sensor binary run the following command from:

vendor/qcom/proprietary/chi-cdk/cdk/tools/

```
python ./buildbins.py
```

Step 3. Locate the generated sensor binary files at *vendor/qcom/proprietary/chi-cdk/vendor/bin/*.

Step 4. Push the sensor binary file to *./usr/lib/camera/* with the appropriate root permissions. The target system loads the XML files that have been converted to binaries for camera software use.

3. Bring up camera sensor

3.1. Configure sensor software

Sensor-specific XML

Every sensor has an associated configuration XML file that defines features, such as power settings, resolution, initialization settings, and exposure settings.

The complete settings are present at the **<sensorDriverData></sensorDriverData>** node of this XML file.

Table 2-1: Sensor information nodes– slaveInfo

The slaveInfo node in the XML file holds the information that is used by the driver while probing the sensor.

Field	Description
slaveInfo	Contains the sensor slave information and power settings.
SensorName	Name of the sensor. **Example**: imx230
slaveAddress	8 bit or 10 bit slave address. **Example**: 32 For 0x20 slave address, the field value is 32.
regAddrType	Register address/data size in bytes. **Example**: 1 = Byte address 2 = Word address 3 = 3 byte address 4 = Address type max
regDataType	Register address/data size in bytes. **Example:** 1 = Byte data 2 = Word data 3 = Double word data 4 = Data type max
sensorIdRegAddr	Register address for sensor ID. **Example**: 22

Thundercomm Confidential

Field	Description
sensorId	Sensor ID. **Example**: 560
sensorIDMask	Mask for sensor ID. Sensor ID may only be a few bits. **Example**: 4294967295
i2cFrequencyMode	I2C Frequency mode of slave. **Example**: FAST Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz) • CUSTOM (custom frequency in DTSI)
powerUpSequence	Contains the power-up configuration sequence required to control the power to the device while turning it on. **Example**: **<powerUpSequence> <powerSetting> <configType>RESET</configType> <configValue>0</configValue> <delayMs>0</delayMs> </powerSetting> </ powerUpSequence>**
powerSetting	Contains power configuration type, value, and delay.
configType	Power configuration type. Example: MCLK Supported types are: • MCLK • VANA • VDIG • VIO • VAF • RESET • STANDBY
configValue	Power configuration type. **Example**: 24000000
delayMs	Delay in milliseconds. **Example**: 1
powerDownSequence	Contains the power-down configuration sequence that is required to control the power to the device while closing it. **Example**: **<powerDownSequence> <powerSetting> <configType>RESET</configType> <configValue>0</configValue> <delayMs>0</delayMs> </powerSetting> </ powerDownSequence>**

Thundercomm Confidential

Table 2-2: Sensor information nodes– refAddrInfo

The register information node contains the configuration register addresses for various sensor features such as to set gain, frame length lines, and test pattern generation.

Field	Description
regAddrInfo	Contains the information about the register addresses for various sensor settings.
xOutput	Controls the program width. **Example**: 844 If the register address is 0x34C, the value of the field will be 844.
yOutput	Controls the program height. **Example**: 846
frameLengthLines	Programs the frame length lines. **Example**: 832
lineLengthPixelClock	Programs the line length pixel clock. **Example**: 834
coarseIntgTimeAddr	Programs the coarse integration time. **Example**: 514
globalGainAddr	Program the gain channel. **Example**: 516
digitalGainRedAddr	Programs digital gain for the red channel. This address is optional but is required if supported by sensor. **Example**: 528
digitalGainGreenRedAddr	Programs digital gain for the green-red channel. This address is optional but is required if supported by sensor. **Example**: 526
digitalGainBlueAddr	Register address to program digital gain for the blue channel. This address is optional but is required if supported by sensor. **Example**: 530
digitalGainGreenBlueAddr	Programs digital gain for the green-blue channel. The address is optional but is required when supported by sensor. **Example**: 532
testPatternRAddr	Programs manual test pattern value for the red channel. **Example**: 1538
testPatternGRAddr	Programs manual test pattern value for the red channel. **Example**: 1540
testPatternBAddr	Programs manual test pattern value for the red channel. **Example**: 1542
testPatternGBAddr	Programs manual test pattern value for the red channel. **Example**: 1544

Table 2-3: Sensor information nodes– resolutionInfo

The resolution information node contains the information regarding resolution settings and configuration.

Thundercomm Confidential

Field	Description
resolutionInfo	This node holds the configuration and settings for all the resolutions.
resolutionData	Holds the configuration data for one resolution. The number of sensor-supported resolutions is equal to the number of resolutionData nodes. First node of the resolutionData should always point to the full resolution configuration of the sensor.
colorFilterArrangement	Color filter arrangement of the sensor. **Example**: BAYER_RGGB Supported filter arrangements are: • BAYER_BGGR • BAYER_GBRG • BAYER_GRBG • BAYER_RGGBn • BAYER_Y • YUV_UYVY • YUV_YUYV

streamInfo	Information related to the stream configuration.
	Example:
	`<streamInfo>`
	`<streamConfiguration>`
	`<vc range="[0,3]">0</vc>`
	`<dt>43</dt>`
	`<frameDimension>`
	`<xStart>0</xStart>`
	`<yStart>0</yStart>`
	`<width>4608</width>`
	`<height>2592</height>`
	`</frameDimension>`
	`<bitWidth>10</bitWidth>`
	`<type>IMAGE</type>`
	`</streamConfiguration>`
	`</streamInfo>`

streamConfiguration	Information related to the stream data.
vc	Virtual channel of the data **Example**: 2 Each unique combination of virtual channel (VC) and DT should map to a unique channel identifier (CID) value. Possible CID values for a given VC are: 0 – 0, 1, 2, 3 1 – 4, 5, 6, 7 2 – 8, 9, 10, 11 3 – 12, 13, 14, 15 Valid values for VC are: 0, 1, 2, and 3
dt	DT of the stream. **Example**: 43 Default value is 0x2B (10 bit RAW)

Thundercomm Confidential
Sent: 2021-1-31 9:16:31 AM CST
sender_ucam.moonvision.com

Field	Description
frameDimension	Frame dimension contains x and y start coordinates, and the total width and height of the image.
xStart	X coordinate of start of image.
yStart	Y coordinate of start of image.
width	Width of the image.
height	Height of the image.
bitWidth	Bit width of the data.
type	Stream type. **Example**: IMAGE Supported stream types are: • BLOB • IMAGE • PDAF • HDR • META
lineLengthPixelClock	Line length pixel clock of frame. Typically, this value is the active width + blanking width.
frameLengthLines	Frame length lines of frame. Typically, this value is the active height + blanking height.
minHorizontalBlanking	Minimum horizontal blanking interval in pixels.
minVerticalBlanking	Minimum horizontal blanking interval in lines.
outputPixelClock	Represents how much data comes out of the camera over MIPI lanes to set the VFE clock. op_pixel_clk = (Total data rate from sensor)/(Bits per pixel) For example, if the MIPI DDR clock value (speed of the clock lane of the MIPI camera sensor) is 300 MHz and the sensor transmits on four lanes, each lane has a 600 MHz data rate. Thus, the total data rate is 2400 MHz. For 10 bits per pixel bayer data, the op_pixel_clk value is 2400/10 = 240 MHz. The values are filled in accordance with the sensor specifications. The values are calculated based on the register settings configured for the camera sensor.
horizontalBinning	Horizontal binning value.
verticalBinning	Vertical binning value.
framerate	Maximum frame rate.
laneCount	The number of data lanes on which the sensor outputs data for a given mode of operation. The maximum data lane capability (given in the datasheet) of the sensor along with the sensor register settings configured in the driver determines the value.

Thundercomm Confidential

Field	Description
settleTimeNs	Settle time in nanoseconds. The value is configured, based on the output characteristics of the sensor, to ensure that the PHY transmitter of the sensor does not have sync issues with the PHY receiver of MSM chipsets. For 28 nm and smaller MSM parts, use the following formula to calculate the settle count: **settle_CNT=(T(HS_SETTLE)_AVG)/T(TIMER_CLK)** where, **T(HS_SETTLE)_avg = (T(HS_SETTLE)_min + T(HS_SETTLE)_max)/2** as indicated by sensor datasheet. TIMER_CLK refers to the operating frequency of PHY interface to which the camera sensor is connected (for example, CAMSS_PHY0_CSI0PHYTIMER_CLK for PHY0). This value is set in the *kernel/arch/arm/boot/dts/msm/msmXXXX-camera.dtsi*, where XXXX refers to the MSM chipset in use. T(TIMER_CLK) is the duration of a clock cycle when the operating frequency is equal to TIMER_CLK, and is represented in nano second units. For example, T(TIMER_CLK) for TIMER_CLK 200 MHz is 【1 × (10^9)】/【200 × (10^6)】 = 5 ns. If the calculated value does not work as expected, try to alternate the lower values between **T(HS_SETTLE)_avg/T(TIMER_CLK)** and **T(HS_SETTLE)_min/T(TIMER_CLK)**. See https://mipi.org/specifications/d-phy for more details on T(HS_SETTLE).
is3Phase	Flag to know if the sensor is a three-phase sensor (C-PHY) or one- phase sensor (D-PHY).
resSettings	Sequence of register settings to configure the device with this resolution. Do not add stream on settings for the sensor in the initialization settings, as this enables the sensor streaming before real software stream on and may cause PHY to miss LP sequence. **Example**: **<resSettings> <regSetting> <registerAddr>0x114</registerAddr> <registerData>0x3</registerData> <regAddrType range="[1,4]">2</regAddrType> <regDataType range="[1,4]">1</regDataType> <operation>WRITE</operation> <delayUs>0x0</delayUs> </regSetting> <resSettings>**
regSetting	This node holds one register configuration and forms a unit of large-resolution register settings sequence.
registerAddr	Register address that is accessed.

Thundercomm Confidential

Field	Description
registerData	If the operation is WRITE, registerData is the data value to be written into the specified register address. If the operation is READ, registerData is the number of bytes to be read from the specified register address.
regAddrType	Type of register address.
regDataType	Type of register data.
operation	Type of operation-supported values: • WRITE • READ • POLL
delayUs	Delay in microseconds. If not explicitly specified, the delay is zero.
cropInfo	Crop information of the frame. **Example**: **<cropInfo> <left>0</left> <right>0</right> <top>0</top> <bottom>0</bottom> </cropInfo>**
left	Left crop pixel information.
right	Right crop pixel information.
top	Top crop pixel information.
bottom	Bottom crop pixel information.
capability	List of features/capabilities supported by the sensor. Supported features are: • HFR • iHDR • INSENSOR_HDR • PDAF • QUADRA_CFA • RAW_HDR • ZIGZAG_HDR

Table 2-4: Sensor information nodes– exposureControlInfo

The exposure control information node contains the exposure details such as maximum gain, maximum line count, and conversion formulas for gain manipulation.

Thundercomm Confidential

Field	Description
exposureControlInfo	This node holds the information about the exposure details such as maximum gain, maximum line count, and conversion formulas for gain manipulation. **Example**: **<exposureControlInfo> <maxAnalogGain>8</maxAnalogGain> <maxDigitalGain>2</maxDigitalGain> <verticalOffset>20</verticalOffset> <maxLineCount>65515</maxLineCount> <realToRegGain>512-(512/realGain)</realToRegGain> <regToRealGain>512/(512-regGain)</regToRealGain> </exposureControlInfo>**
maxAnalogGain	Maximum analog gain supported by the sensor.
maxDigitalGain	Maximum digital gain supported by the sensor.
verticalOffset	Minimum offset to be maintained between the line count and frame length lines.
maxLineCount	Maximum line count supported by the sensor.
realToRegGain	Real gain to register the gain equation. The equation must contain realGain in the equation.
regToRealGain	Register gain to real gain equation. The equation must contain regGain in the equation.

Table 2-5: Sensor information nodes– streamOnSettings

Register settings to start streaming.

Field	Description
streamOnSettings	Sequence of register settings to configure the device to start streaming. In the sensor driver, the following settings are configured to match this guideline: **NOTE**: Take the clock and data lanes from LP11 to HS TxState. **Example**: **<streamOnSettings> <regSetting> <registerAddr>0x0100</registerAddr> <registerData>0x1</registerData> <regAddrType range="[1,4]">2</regAddrType> <regDataType range="[1,4]">1</regDataType> <operation>WRITE</operation> <delayUs>0</delayUs> </regSetting> </streamOnSettings>**
regSetting	Register setting configuration. Contains: Register address, register data, register address type, register DT, operation, and delay in micro seconds

Thundercomm Confidential

Field	Description

streamOffSettings	Sequence of register settings to configure the device to stop streaming. Configure the following settings to match this guideline and take the clock and data lanes to LP11.
	Example:
	`<streamOffSettings>`
	`<regSetting>`
	`<registerAddr>0x0100</registerAddr>`
	`<registerData>0x0</registerData>`
	`<regAddrType range="[1,4]">2</regAddrType>`
	`<regDataType range="[1,4]">1</regDataType>`
	`<operation>WRITE</operation>`
	`<delayUs>0</delayUs>`
	`</regSetting>`
	`</streamOffSettings>`

regSetting	Register setting configuration. Contains: Register address, register data, register address type, register DT, operation, and delay in micro seconds

Table 2-6: Sensor information nodes– groupHoldOnSettings

Register settings to configure the device in group hold settings.

Field	Description

groupHoldOnSettings	Sequence of register settings to configure the device in group hold settings.
	Example:
	`<groupHoldOnSettings>`
	`<regSetting>`
	`<registerAddr>0x0104</registerAddr>`
	`<registerData>0x1</registerData>`
	`<regAddrType range="[1,4]">2</regAddrType>`
	`<regDataType range="[1,4]">1</regDataType>`
	`<operation>WRITE</operation>`
	`<delayUs>0</delayUs>`
	`</regSetting>`
	`</groupHoldOnSettings>`

regSetting	Register setting configuration. Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.

Thundercomm Confide

Field	Description
groupHoldOffSettings	Sequence of register settings to configure the device to stop group hold settings. **Example**: **<groupHoldOffSettings> <regSetting> <registerAddr>0x0104</registerAddr> <registerData>0x0</registerData> <regAddrType range="[1,4]">2</regAddrType> <regDataType range="[1,4]">1</regDataType> <operation>WRITE</operation> <delayUs>0</delayUs> </regSetting> </groupHoldOffSettings>**
regSetting	Register setting configuration. Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.

Table 2-7: Sensor information nodes– initSettings

Sequence of register settings to initialize the sensor.

Field	Description
initSettings	Sequence of register settings to initialize the sensor. Do not add stream on settings for the sensor in the initialization settings, as this enables the sensor streaming before real software stream on and may cause PHY to miss LP sequence. **Example**: **<initSettings> <regSetting> <registerAddr>0x136</registerAddr> <registerData>0x18</registerData> <regAddrType range="[1,4]">2</regAddrType> <regDataType range="[1,4]">1</regDataType> <operation>WRITE</operation> <delayUs>0</delayUs> </regSetting> </initSettings>**
regSetting	Register setting configuration. Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.

Table 2-8: Sensor information nodes– testPatternInfo

This node contains the information about the test pattern generation register settings.

Field	Description
testPatternInfo	Contains the information about the test pattern generation register settings. **Example**: **<testPatternInfo> <testPatternData> <mode>OFF</mode> <settings> <regSetting> <registerAddr>0x136</registerAddr> <registerData>0x18</registerData> <regAddrType range="[1,4]">2</regAddrType> <regDataType range="[1,4]">1</regDataType> <operation>WRITE</operation> <delayUs>0</delayUs> </regSetting> </settings> </testPatternData> </testPatternInfo>**

Table 2-9: Sensor information nodes– testPatternData

Field	Description
testPatternData	This node holds the register and mode settings of a particular test pattern.
Mode	Supported modes are: • OFF • SOLID_COLOR • COLOR_BARS • COLOR_BARS_FADE_TO_GRAY • PN9 • CUSTOM1
settings	Sequence of register settings to configure the test pattern on the sensor.
regSetting	Register setting configuration. Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.

Table 2-10: Sensor information nodes– colorLevelInfo

The color level information node provides details about the various channels in complete dark light.

Field	Description
colorLevelInfo	Color level information. The default currents in various channels in complete dark light. **Example**: **<colorLevelInfo> <whiteLevel>1023</whiteLevel> <rPedestal>64</rPedestal> <grPedestal>64</grPedestal> <bPedestal>64</bPedestal> <gbPedestal>64</gbPedestal> </colorLevelInfo>**

whiteLevel	White level value.
rPedestal	Pedestal value for the red channel.
grPedestal	Pedestal value for the green-red channel.
bPedestal	Pedestal value for the blue channel.
gbPedestal	Pedestal value for the green-blue channel.

Table 2-11: Sensor information nodes– Black region

Field	Description
opticalBlackRegionInfo	Information about black regions. Multiple black regions are provided, if applicable. **Example**: **<opticalBlackRegionInfo> <dimension> <xStart>0</xStart> <yStart>0</yStart> <width>4608</width> <height>2592</height> </dimension> </opticalBlackRegionInfo>**
dimension	Frame dimension: Contains xStart, yStart, width, and height.
xStart	X start coordinate of the region.
yStart	Y start coordinate of the region.
width	Width of the region.
height	Height of the region.

Table 2-12: Sensor information nodes– Pixel array

Field	Description
pixelArrayInfo	Information about the pixel array. Active dimension and dummy pixels width are provided.
activeDimension	Width and height of the frame or subframe. **Example**: **<activeDimension> <xStart>0</xStart> <yStart>0</yStart> <width>4608</width> <height>2592</height> </activeDimension>**
xStart	Start of the X output in the frame.
yStart	Start of the Y output in the frame.
width	Total width of the frame in pixels.
height	Total height of the frame in pixels.

Thundercomm Confidential

Field	Description
dummyInfo	Dummy pixels surrounding the active pixel array. **Example**: **<dummyInfo> <left>0</left> <right>0</right> <top>0</top> <bottom>0</bottom> </dummyInfo>**
left	Starting coordinate of the left dummy pixel.
right	Starting coordinate of the right dummy pixel.
top	Starting coordinate of the top dummy pixel.
bottom	Starting coordinate of the bottom dummy pixel.

Table 2-13: Sensor information nodes– Delay

Field	Description
appDelayInfo	Delay information.
linecount	Number of frames required to apply the line count.
gain	Number of frames required to apply the gain.
maxPipeline	Maximum pipeline delay in number of frames.
frameSkip	Number of initial bad frames to skip.

Table 2-14: Sensor information nodes- Sensor property

Field	Description
sensorProperty	Sensor property information. **Example**: **<sensorProperty> <pixelSize>0</pixelSize> <cropFactor>0</cropFactor> <sensingMethod>ONE_CHIP_COLOR_AREA</sensingMethod> </sensorProperty>**
pixelSize	Pixel size in micro meters.
cropFactor	Crop factor.
sensingMethod	Sensing method of sensor. Supported sensing methods: • UNDEFINED • ONE_CHIP_COLOR_AREA TWO_CHIP_COLOR_AREA • THREE_CHIP_COLOR_AREA • COLOR_SEQUENCE_AREA • TRILINEAR • COLOR_SEQUENCE_LINEAR

Module configuration XML

This XML is used to store the camera module-specific information, such as the lens information, mount angles, actuator, OIS, and flash-related information.

Thundercomm Confidential

Table 2-15: CameraPosition

Field	Description
CameraPosition	Position of the sensor module. Supported values are: • REAR • FRONT • REAR_AUX • FRONT_AUX • EXTERNAL

Table 2-16: LensInformation

Field	Description
LensInformation	This node holds sensor lens-specific information.
focalLength	Focal length of the lens in millimeters.
fNumber	F-Number of the optical system.
minFocusDistance	Minimum focus distance in meters.
maxFocusDistance	Total focus distance in meters.
horizontalViewAngle	Horizontal view angle in degrees.
verticalViewAngle	Vertical view angle in degrees.
maxRollDegree	Maximum roll degree.
maxPitchDegree	Maximum pitch degree.
maxYawDegree	Maximum yaw degree.

Table 2-17: LensInformationList

Field	Description
LensInformationList	This node holds sensor list of lens-specific information based on version number.
version	This corresponds to module version defined in EEPROM.
chromatixName	Chromatix name corresponding to this version.
lensInfo	Lens specific information. See Table 2-16.

Table 2-18: ModuleConfiguration

Field	Description
ModuleConfiguration	Module configuration.
cameraId	cameraId is the ID to which DTSI node is mapped. Typically, cameraId is the slot ID for Non-Combo mode.
moduleName	Name of the module integrator.
sensorName	Name of the sensor in the image sensor module.
sensorSlaveAddress	Sensor slave address to override the value present in the binary. Optional element. Do not use this entry if override is not required.

Thundercomm Confidential

Field	Description
sensorI2CFrequencyMode	Sensor operation frequency mode. Supported types are: • STANDARD • FAST • FAST_PLUS • CUSTOM
actuatorName	Name of the actuator in the image sensor module. Optional element. Skip this element if actuator is not present.
oisName	Name of the OIS in the image sensor module. Optional element. Skip this element if OIS is not present.
eepromName	Name of the EEPROM in the image sensor module. Optional element. Skip this element if EEPROM is not present.
flashName	Flash name to open binary. Binary name is of form *flashName_flash.bin Ex:-pmic_flash.bin*.
chromatixName	To open binary. Binary name is of the form *sensor_model_chromatix.bin*.
pdafName	Name of the PDAF driver used to configure this image sensor module. Optional element. Skip this element if PDAF is not supported.
position	Camera position. Supported types are: • REAR • FRONT • REAR_AUX • FRONT_AUX • EXTERNAL
isComboMode	Flag to enable Combo mode. This flag is enabled if multiple sensors are using same CSI-PHY receiver: • First camera – Connected to PHY lanes 2:0. Clock lane is connected to PHY lane 1. Data lanes (up to 2) are connected to either of PHY lanes 0 or 2. • Second camera – Connected to PHY lanes 4:3. Clock lane is connected to PHY lane 4. Data lanes are connected to PHY lane 3.
lensInfoList	List of lens information nodes. See Table 2-16.

Table 2-19: ModuleGroup

Field	Description
ModuleGroup	Module group can contain either one module or two odules. Dual camera, stereo camera use cases contain two odules in the group.
moduleConfiguration	Module configuration information. See Table 2-18.

Thundercomm Confidential

Table 2-20: CameraModuleData

Field	Description
CameraModuleData	Camera module data.
module_version	Camera module version.
major_revision	Contains the driver revision.
minor_revision	Contains the minor revision number.
incr_revision	Contains incremental revision number.
moduleGroup	Specifies the module configuration data (see Table 2-18) count, ID, and the data (see Table 2-20).

3.2. Sensor hardware configuration

The hardware configuration of the camera sensor module is saved in the kernel DTSI files.

3.3. Sensor kernel nodes

Table 2-21: Camera_LDO

Field	Description
compatible	Data type to which this node corresponds.
reg	ID of the node.
regulator-name	LDO regulator name.
regulator-min-microvolt	Minimum voltage for this LDO.
regulator-max-microvolt	Maximum voltage for this LDO.
regulator-enable-ramp-delay	The time taken, in microseconds, for the supply rail to reach the target voltage, ± whatever tolerance the board design requires. This property describes the total system ramp time required due to the combination of internal ramping of the regulator itself, and board design issues such as trace capacitance and load on the supply.
enable-active-high	Enable with active high.
gpio	GPIO to be used with this node.
pinctrl-names	Pinctrl name for this GPIO.
pinctrl-0	Pinctrl handle for this GPIO.
vin-supply	Input voltage supply node name.

Example:

```
camera_ldo: gpio-regulator@2 {compatible = "regulator-fixed"; reg = <0x02 0x00>;
regulator-name = "camera_ldo"; regulator-min-microvolt = <1050000>;
regulator-max-microvolt = <1050000>;
regulator-enable-ramp-delay = <233>; enable-active-high;
gpio = <&pm8998_gpios 9 0>; pinctrl-names = "default";
pinctrl-0 = <&camera_dvdd_en_default>; vin-supply = <&pm8998_s3>;
};
```

Thundercomm Confidential

Table 2-22: CCI-based sensor driver

Field	Description
cell-index	Points to the ID of the sensor node.
reg	Corresponds to the cell-index of the node.
compatible	Data type to which this node corresponds to.
qcom,csiphy-sd-index	Paired CSIPHY node for this DT.
qcom,sensor-position-roll	Sensor position roll angle.
qcom,sensor-position-pitch	Sensor position pitch angle.
qcom,sensor-position-yaw	Sensor position yaw angle.
qcom,led-flash-src	LED flash source node name.
qcom,actuator-src	Actuator source node name.
qcom,ois-src	OIS source node name.
qcom,eeprom-src	EEPROM source node name.
cam_vio-supply	I/O voltage supply source.
cam_vana-supply	Analog voltage source.
cam_vdig-supply	Digital voltage source.
cam_clk-supply	Camera clock source.
qcom,cam-vreg-name	Voltage regulator node names.
qcom,cam-vreg-min-voltage	Minimum voltage, in the above order.
qcom,cam-vreg-max-voltage	Maximum voltage, in the above order.
qcom,cam-vreg-op-mode	Operation mode of the voltage regulator, in above order.
qcom,gpio-no-mux	Whether GPIO muxing is enabled.
pinctrl-names	Pinctrl handle names.
pinctrl-0	Binding to GPIO pin and function node.
pinctrl-1	Binding to GPIO pin and function node.
gpios	List of GPIO pins used.
qcom,gpio-reset	Should contain index to GPIO used by sensors reset_n.
qcom,gpio-vana	Should contain index to GPIO used by sensors analog vreg enable.
qcom,gpio-vaf	Should contain index to GPIO used by sensors of vreg enable.
qcom,gpio-req-tbl-num	Should contain index to GPIO specific to this sensor.
qcom,gpio-req-tbl-flags	Should contain direction of GPIO present in qcom, gpio- req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Should contain name of GPIO present in qcom, gpio-req- tbl-num property (in the same order).
qcom,sensor-position	Mount angle of the sensor: • 0 – Back Camera • 1 – Front Camera

Thundercomm Confidential

Field	Description
qcom,sensor-mode	Supported sensor mode: • 0 – back camera 2D • 1 – front camera 2D • 2 – back camera 3D • 3 – back camera int 3D
qcom,cci-master	I2C master used for this sensor: • 0 – MASTER 0 • 1 – MASTER 1
status	Whether this node is enabled or disabled.
clocks	Clock node names.
clock-names	Name of the clocks required for the device.
qcom,clock-rates	Clock rate in Hz.

Example:

```
qcom,cam-sensor@0 { cell-index = <0>;
compatible = "qcom,cam-sensor"; reg = <0x0>;
qcom,csiphy-sd-index = <0>;
qcom,sensor-position-roll = <90>;
qcom,sensor-position-pitch = <0>;
qcom,sensor-position-yaw = <180>; qcom,led-flash-src = <&led_flash_rear>;
qcom,actuator-src = <&actuator_rear>; qcom,ois-src = <&ois_rear>;
qcom,eeprom-src = <&eeprom_rear>; cam_vio-supply = <&pm8998_lvs1>;
cam_vana-supply = <&pmi8998_bob>; cam_vdig-supply = <&camera_rear_ldo>;
cam_clk-supply = <&titan_top_gdsc>;
qcom,cam-vreg-name = "cam_vio", "cam_vana", "cam_vdig", "cam_clk";
qcom,cam-vreg-min-voltage = <0 3312000 1050000 0>;
qcom,cam-vreg-max-voltage = <0 3600000 1050000 0>;
qcom,cam-vreg-op-mode = <0 80000 105000 0>;
qcom,gpio-no-mux = <0>;
pinctrl-names = "cam_default", "cam_suspend"; pinctrl-0 = <&cam_sensor_mclk0_active
&cam_sensor_rear_active>; pinctrl-1 = <&cam_sensor_mclk0_suspend
&cam_sensor_rear_suspend>; gpios = <&tlmm 13 0>,
<&tlmm 80 0>, <&tlmm 79 0>;
qcom,gpio-reset = <1>;
qcom,gpio-vana = <2>;
qcom,gpio-req-tbl-num = <0 1 2>;
qcom,gpio-req-tbl-flags = <1 0 0>; qcom,gpio-req-tbl-label = "CAMIF_MCLK0",
"CAM_RESET0", "CAM_VANA";
qcom,sensor-mode = <0>;
qcom,cci-master = <0>; status = "ok";
clocks = <&clock_camcc CAM_CC_MCLK0_CLK>; clock-names = "cam_clk";
qcom,clock-rates = <24000000>;
};
```

Table 2-23: OIS node

Field	Description
cell-index	Points to the ID of this node.
reg	Corresponds to the cell-index of the node.
compatible	Data type to which this node corresponds to.
qcom,cci-master	CCI node that drives this node.
cam_vaf-supply	Should contain regulator from which AF voltage is supplied.
qcom,cam-vreg-name	Voltage regulator node names.
qcom,cam-vreg-min-voltage	Minimum voltage, in the above order.
qcom,cam-vreg-max-voltage	Maximum voltage, in the above order.
qcom,cam-vreg-op-mode	Operation mode of the voltage regulator, in above order.
status	Whether this node is enabled or disabled.

Example:

```
ois_rear: qcom,ois@0 { cell-index = <0>; reg = <0x0>;
compatible = "qcom,ois"; qcom,cci-master = <0>;
cam_vaf-supply = <&actuator_regulator>; qcom,cam-vreg-name = "cam_vaf";
qcom,cam-vreg-min-voltage = <2800000>;
qcom,cam-vreg-max-voltage = <2800000>;
qcom,cam-vreg-op-mode = <0>; status = "disabled";
};
```

Table 2-24: EEPROM node

Field	Description
cell-index	Points to the node ID.
reg	Corresponds to the cell-index of the node.
compatible	Data type that this node corresponds to.
qcom,cci-master	CCI node that drives this node.
cam_vio-supply	Input/output voltage supply source.
cam_vana-supply	Analog voltage source.
cam_vdig-supply	Digital voltage source.
cam_clk-supply	Camera clock source.
qcom,cam-vreg-name	Voltage regulator node names.
qcom,cam-vreg-min-voltage	Minimum voltage, in the above order.
qcom,cam-vreg-max-voltage	Maximum voltage, in the above order.
qcom,cam-vreg-op-mode	Operation mode of the voltage regulator, in above order.
qcom,gpio-no-mux	Whether GPIO muxing is enabled.
pinctrl-names	Pinctrl handle names.
pinctrl-0	Binding to GPIO pin and function node.
pinctrl-1	Binding to GPIO pin and function node.
gpios	List of GPIO pins used.
qcom,gpio-reset	Should contain index to GPIO used by sensors reset_n.

Thundercomm confidential

Field	Description
qcom,gpio-vana	Should contain index to GPIO used by sensors analog vreg enable.
qcom,gpio-vaf	Should contain index to GPIO used by sensors i2af vreg enable.
qcom,gpio-req-tbl-num	Should contain index to GPIO specific to this sensor.
qcom,gpio-req-tbl-flags	Should contain direction of GPIO present in qcom, gpio-req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Should contain name of GPIO present in qcom, gpio-req-tbl-num property (in the same order).
qcom,sensor-position	Mount angle of the sensor 0 – Back camera 1 – Front camera
qcom,sensor-mode	Supported sensor mode 0 – Back camera 2D 1 – Front camera 2D 2 – Back camera 3D 3 – Back camera int 3D
qcom,cci-master	I2C master used for this sensor 0 – MASTER 0 1 – MASTER 1
status	Whether this node is enabled or disabled.
clocks	Clock node names.
clock-names	Name of the clocks required for the device.
qcom,clock-rates	Clock rate in Hz.

Example:

```
eeprom_rear: qcom,eeprom@0 { cell-index = <0>;
reg = <0>;
compatible = "qcom,eeprom"; cam_vio-supply = <&pm8998_lvs1>; cam_vana-
supply = <&pmi8998_bob>;
cam_vdig-supply = <&camera_rear_ldo>; cam_clk-supply = <&titan_top_gdsc>;
qcom,cam-vreg-name = "cam_vio", "cam_vana", "cam_vdig", "cam_clk";
qcom,cam-vreg-min-voltage = <0 3312000 1050000 0>;
qcom,cam-vreg-max-voltage = <0 3600000 1050000 0>;
qcom,cam-vreg-op-mode = <0 80000 105000 0>;
qcom,gpio-no-mux = <0>;
pinctrl-names = "cam_default", "cam_suspend"; pinctrl-0 =
<&cam_sensor_mclk0_active
&cam_sensor_rear_active>; pinctrl-1 = <&cam_sensor_mclk0_suspend
&cam_sensor_rear_suspend>; gpios = <&tlmm 13 0>,
<&tlmm 80 0>,
<&tlmm 79 0>,
<&tlmm 27 0>;
qcom,gpio-reset = <1>;
qcom,gpio-vana = <2>;
qcom,gpio-vaf = <3>;
qcom,gpio-req-tbl-num = <0 1 2 3>;
qcom,gpio-req-tbl-flags = <1 0 0 0>; qcom,gpio-req-tbl-label =
"CAMIF_MCLK0",
"CAM_RESET0", "CAM_VANA0", "CAM_VAF";
qcom,sensor-position = <0>;
qcom,sensor-mode = <0>;
```

```
qcom,cci-master = <0>; status = "ok";
clocks = <&clock_camcc CAM_CC_MCLK0_CLK>; clock-names = "cam_clk";
qcom,clock-rates = <24000000>;
};
```

3.4. Configure CCI operation speed

Following the I2C specification, CCI (for chipsets where it is present) can operate on the following frequencies:

- 100 kHz (STANDARD)
- 400 kHz (FAST)
- 1 MHz (FAST PLUS)

I2C Frequency mode controls the operation speed. For Custom mode, the CCI tuning parameters setting information is in *../arm/boot/dts/qcom/ XXX-camera.dtsi*.

Example:

```
i2c_freq_custom: qcom,i2c_custom_mode { qcom,hw-thigh = <38>;
qcom,hw-tlow = <56>;
qcom,hw-tsu-sto = <40>;
qcom,hw-tsu-sta = <40>;
qcom,hw-thd-dat = <22>;
qcom,hw-thd-sta = <35>;
qcom,hw-tbuf = <62>;
qcom,hw-scl-stretch-en = <1>;
qcom,hw-trdhld = <6>;
qcom,hw-tsp = <3>;
qcom,cci-clk-src = <37500000>; status = "ok";
};
```

If a custom CCI configuration is used, then the speed of I2C frequency information is calculated by the formula CCI clock = (src clock)/(hw_thigh+ hw_tlow). As the CCI clock frequency is typically 19.2 MHz, the standard CCI frequency case is: CCI clock = 19.2 MHz/(78 + 114) = 100 kHz. If there is no CCI hardware, or when QUP is used for I2C, QUP speed is set to either 100 kHz or 400 kHz from the DTSI file.

3.5. Use different interfaces

The serial peripheral interface (SPI) is not supported for sensor configuration. It only supports the EEPROM driver. For chipsets that do not have CCI hardware, QUP is used to perform I2C transactions with camera.

3.6. Power regulator configuration

The type of voltage to be applied is specified in driver XML for a given sensor. The supported values are:

- MCLK – mclock
- VANA – Analog supply voltage
- VDIG – Digital supply voltage
- VIO – I/O supply voltage
- VAF – Actuator supply voltage
- RESET – Reset GPIO
- STANDBY – Standby GPIO

The values maps to specific voltage regulators using the DTSI sensor node.

PMIC-based sensor nodes

For example, VIO maps to cam_vio-supply, VANA maps to cam_vana-supply, and MCLK maps to cam_clk-supply.

```
cam_vio-supply = <&pm8998_lvs1>; cam_vana-supply = <&pmi8998_bob>;
cam_vdig-supply = <&camera_rear_ldo>; cam_clk-supply = <&titan_top_gdsc>;
```

GPIO-based sensor nodes

```
pinctrl-names = "cam_default", "cam_suspend"; pinctrl-0 =
<&cam_sensor_mclk0_active
&cam_sensor_rear_active>; pinctrl-1 = <&cam_sensor_mclk0_suspend
&cam_sensor_rear_suspend>; gpios = <&tlmm 13 0>,
<&tlmm 80 0>,
<&tlmm 79 0>;
gpio-reset = <1>;
gpio-vana = <2>;
gpio-req-tbl-num = <0 1 2>;
gpio-req-tbl-flags = <1 0 0>;
gpio-req-tbl-label = "CAMIF_MCLK0","CAM_RESET0","CAM_VANA";
```

Regulator-based sensor nodes

```
regulator-names = "cam_vio", "cam_vana", "cam_vdig","cam_clk"; rgltr-cntrl-support;
rgltr-min-voltage = <0 3312000 1050000 0>;
rgltr-max-voltage = <0 3600000 1050000 0>;
rgltr-load-current = <0 80000 105000 0>;
```

Customers can configure different power regulators and supply using the DTSI node parameters.

3.7. Clock configuration

In the DTS file, for each sensor node, the customer can configure clock source as follows:

```
clocks = <&clock_camcc CAM_CC_MCLK0_CLK>; clock-names = "cam_clk";
clock-cntl-level = "turbo"; clock-rates = <24000000>;
```

The order of the lists in the two properties is important. The nth clock-name corresponds to the nth entry in the clock property. The customer does not need to change the list order, as it is parsed in the clock framework. Support is provided for up to four CAM_MCLK running at 19.2 MHz by default.

Thundercomm Confidential

- CAM_CLK peak-to-peak jitter < 400 ps with default frequency.
- Recommended CAM_MCLK for QCS610/QCS410 is 19.2 MHz.

Discuss with Qualcomm Customer Engineering before using any CAM_MCLK frequency other than 19.2 MHz to investigate if jitter is acceptable for such case.

3.8. Power regulator configuration

Sensor infrastructure provides an interface for OEM to write their own gain/exposure update functions. With this change, each sensor can have its own custom library, which implements the API exposed in camxsensordriverAPI.h. After the API methods are implemented, generate the library with the name com.<vendor name>.sensor.<sensor name>.so. This file should also be included in the device-vendor.mk file with the other library files that need to be generated.

Generated library is available at etc/camera/. Once the library with the specified syntax name in the specified path is available, sensor software module loads the binary and accesses the API methods to use for exposure-related functionality.

Sample library implementation is available for imx318 sensor at: *vendor/qcom/proprietary/chi-cdk/vendor/sensor/default/imx318/*.

This infrastructure also allows data published using the vendor tag org.quic.camera2.sensor_register_control to read and pass it to sensor library as SensorFillExposureData->additionalInfo.

Library functions

It is possible to define an addition to the sensor driver using a library. This library can only contain gain-specific functions with the following definitions: static double RegisterToRealGain(unsigned int regGain); static unsigned int RealToRegisterGain(double realGain);

Example:

```
static double RegisterToRealGain(unsigned int regGain){ double realGain;
if(regGain > SENSOR_MAkX_AGAIN_REG_VAL){ regGain = IMX318_MAX_AGAIN_REG_VAL;
}
realGain = 512.0 / (512.0 - regGain); return realGain;
}
static unsigned int RealToRegisterGain(double realGain){ unsigned int
regGain = 0;
if (realGain < IMX318_MIN_AGAIN){ realGain = SENSOR_MIN_AGAIN;
}
regGain = (unsigned int)(512.0 - (512.0 / realGain)); return regGain;
}
void GetSensorLibraryAPIs(SensorLibraryAPI*
pSensorLibraryAPI){ pSensorLibraryAPI->pRealToRegisterGain =
RealToRegisterGain; pSensorLibraryAPI->pRegisterToRealGain =
RegisterToRealGain;
}
```

Blanking requirements

The following are blanking requirements for the ISP:

Thundercomm Confidential

- Minimum horizontal blanking – 64
- Minimum vertical blanking – 32

Probe device:

```
Probe:
09-18 08:03:28.897 0 0 I CAM_INFO: CAM-CPAS: cam_cpas_hw_start: 1021 AXI client=[3][cci][0] comp[1024], comp_ab[1024], uncomp[1024]
09-18 08:03:28.908 0 0 I CAM_INFO: CAM-SENSOR: cam_sensor_driver_cmd: 735 Probe success,slot:0,slave_addr:0x34,sensor_id:0x577
09-18 08:03:28.998 0 0 I CAM_INFO: CAM-CPAS: cam_cpas_hw_start: 1021 AXI client=[3][cci][0] comp[1024], comp_ab[1024], uncomp[1024]

match id:
09-18 08:03:22.571 0 0 I CAM_INFO: CAM-CPAS: cam_cpas_hw_start: 1021 AXI client=[3][cci][0] comp[1024], comp_ab[1024], uncomp[1024]
09-18 08:03:22.585 0 0 I CAM_DBG : CAM-SENSOR: cam_sensor_match_id: 640: read id: 0x577 expected id 0x577:
```

4. EEPROM software configuration

4.1. EEPROM-specific XML

EEPROM has an associated configuration XML file used to define calibration parameters such as PDAF, LSC, AF, dual camera, and WBC data. The entire settings are enclosed in the < **EEPROMDriverData** ></ **EEPROMDriverData**> node of this XML.

4.2. EEPROM information node

slaveInfo node

slaveInfo contains the information that is used by the driver to power on/off the EEPROM device. Configuring this node is optional in kernel probe as in kernel probe EEPROM device tree node contains this information.

Table 2-25: EEPROM slaveInfo

Field	Description
slaveInfo	Contains the sensor slave information and power settings.
EEPROMName	Name of the EEPROM. **Example**: atmel_at24c32e
slaveAddress	8-bit or 10-bit slave address. Example: 0xa0

Thundercomm Confi

Field	Description
regAddrType	Register address/data size in bytes. **Example**: • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register address/data size in bytes. **Example**: • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = Data type max
i2cFrequencyMode	I2C Frequency mode of slave. **Example**: FAST Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz) • CUSTOM (custom frequency in DTSI)
powerUpSequence	Contains the power-up configuration sequence required to control the power to the device while turning it on. powerSetting: Contains power configuration type, value, and delay. configType: Power configuration type Supported types are: • MCLK • VANA • VDIG • VIO • VAF • RESET • STANDBY configValue: Value for the specified type or 0 to use default value. delayMs: Delay in milliseconds. **Example**: **<powerUpSequence> <powerSetting> <configType>VIO</configType> <configValue>0</configValue> <delayMs>0</delayMs> </powerSetting> </powerUpSequence>**
powerDownSequence	Contains the power-down configuration sequence required to control the power to the device while turning it off. Field descriptions are same as powerUpSequence.

memoryMap node

Thundercomm Confidential

This memory map node contains the information that is needed by the driver to read the OTP data from EEPROM. Configuring this node is optional in kernel probe as in kernel probe EEPROM device tree node contains this information but at minimum one set of nodes needs to be configured containing READ operation and total size of the data in registerData. To read total size of the data for multiple sets of regSetting nodes, information will be calculated as the sum off all the registerData values corresponding to the READ operation.

Table 2-26: EEPROM memoryMap

Field	Description
memoryMap	Contains regSetting node which contain details for read, write and poll operations. This can contain multiple sets of regSetting nodes.
regSetting	regSetting contains the slaveAddr, registerAddr, registerData, regAddrType, regDataType, operation, delayUS. **Example**: **<regSetting> <slaveAddr>0xa0</slaveAddr> <registerAddr>0x00</registerAddr> <registerData>700</registerData> <regAddrType >2</regAddrType> <regDataType>1</regDataType> <operation>READ</operation> <delayUs>0</delayUs> </regSetting> <regSetting> <slaveAddr>0xa1</slaveAddr> <registerAddr>0x00</registerAddr> <registerData>800</registerData> <regAddrType >2</regAddrType> <regDataType>1</regDataType> <operation>READ</operation> <delayUs>0</delayUs> </regSetting>**
slaveAddr	Slave address to communicate with the device. 8-bit or 10-bit slave address. • **Example**: 0xa0
registerAddr	Register address that is accessed. • **Example**: 0x00
registerData	Register data to be programmed/read. • For WRITE operation, registerData is the data value to be written into the specified register address. **Example**: 0x02 • For READ operation, registerData is the number of bytes to be read from the specified register address. **Example**: 1584 • For POLL operation, registerData is the value for which poll is being done. **Example**: 0x01

Field	Description
regAddrType	Register address type/data size in bytes. **Example**: • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type/data size in bytes. **Example**: • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
operation	Operation to be performed on EEPROM. Supported operations are: • READ • WRITE • POLL
delayUs	Delay in micro seconds. **Example**: 100

formatInfo node

The format node contains the information needed to format the OTP data obtained from EEPROM. It is not necessary to have all the sub fields filled as each EEPROM can have some or all the type of the OTP data and only those, whose data is available can be configured. In the absence of format data information, raw (unformatted) data will be published to meta data pool and that can be used to format and obtain the required data by individual modules.

For formatting the data buffer read from EEPROM following are the requirements:

- The position of each field also known as offset (with starting position reference as 0).
- Exact number of bits of data of that specific field, that is, size of the field represented in the form of mask and endianness to know the byte reading order.

In addition to the raw buffer formatting, this structure can also contain other constants that are not part of the buffer data to be configured.

Table 2-27: EEPROM formatInfo

Field	Description
AF	Auto focus (AF) data information to format the AF OTP data.
autoFocusData	isAvailable – Indicates that AF data is available or not in the OTP data buffer and valid values are true/false. If this is false, all the remaining fields under AF node will be ignored. **Example**: true endianness – Type of the endianness. valid values are BIG or LITTLE Example: LITTLE

Thundercomm Confidential

Field	Description
macro	offset – Position of the first byte of macro value in the buffer. • **Example**: 22 mask – Represents the number of valid bits of the data read from offset • **Example**: 0xFFF. This means 2 bytes of data to be read and in that 12 bits contain the valid data. Mask 0 indicates that particular data is not available in the buffer. The same explanation holds good for all other fields, which needs offset and mask information.
infinity	offset – Position of the first byte of infinity value in the buffer. mask – Represents the number of valid bits of the data.
hall	offset – Position of the first byte of hall value in the buffer. mask – Represents the number of valid bits of the data.
hallBias	offset – Position of the first byte of hall bias value in the buffer mask – Represents the number of valid bits of the data
hallRegisterAddr	Register address for hall and hall bias to be programmed. • **Example**: 0x28
verticalMacro	offset – Position of the first byte of vertical macro value in the buffer. mask – Represents the number of valid bits of the data.
horizontalMacro	offset – Position of the first byte of horizontal macro value in the buffer. mask – Represents the number of valid bits of the data.
verticalInfinity	offset – Position of the first byte of vertical infinity value in the buffer. mask – Represents the number of valid bits of the data.
horizontalInfinity	offset – Position of the first byte of horizontal infinity value in the buffer. mask – Represents the number of valid bits of the data.
macroMargin	Margin to be extended towards macro region. • **Example**: 0.3 that is, the new marco value is macro = macro + DACRange × 0.3 where, DACRange = macro - infinity
infinityMargin	Margin to be extended towards infinity region. • **Example**: 0.15 that is, the new infinity value is infinity = infinity + DACRange × 0.15 where, DACRange = macro - infinity
WB	White balance (WB) data information to format WB data
WBData	isAvailable – Indicates that WB data is available or not in the OTP data buffer and valid values are true/false. If this is false, all the remaining fields under WB node will be ignored. • **Example**: true endianness – Type of the endianness. valid values are BIG or LITTLE Example: LITTLE
dataType	Indicates whether the WB data is available in the form of RATIO or INDIVIDUAL. If the data is available in the form of RATIO, then rOverGValue, bOverGValue, grOverGBValue will be configured. If the data is available in the form of INDIVIDUAL, then rValue, grValue, bValue, gbValue will be configured.

Thundercomm Confidential

Field	Description
lightInfo	Contains the type of the illuminant and corresponding color values based on dataType.
illuminantType	Type of the illuminant for which WB data is available. Supported illuminants are D65, TL84, A, D50, H.
rValue	offset – Position of the first byte of rValue value in the buffer. mask – Represents the number of valid bits of the data.
grValue	offset – Position of the first byte of grValue value in the buffer. mask – Represents the number of valid bits of the data.
bValue	offset – Position of the first byte of bValue value in the buffer. mask – Represents the number of valid bits of the data.
gbValue	offset – Position of the first byte of gbValue value in the buffer. mask – Represents the number of valid bits of the data.
rOverGValue	offset – Position of the first byte of rOverGValue value in the buffer. mask – Represents the number of valid bits of the data.
bOverGValue	offset – Position of the first byte of bOverGValue value in the buffer. mask – Represents the number of valid bits of the data.
grOverGBValue	offset – Position of the first byte of grOverGBValue value in the buffer. mask – Represents the number of valid bits of the data.
mirror	offset – Position of the first byte of mirror value in the buffer. mask – Represents the number of valid bits of the data.
flip	offset – Position of the first byte of flip value in the buffer. mask – Represents the number of valid bits of the data.
qValue	Factor to convert the obtained real values to required float values.
isInvertGROverGB	Set to true if GRoverGB needs to be inverted or else false.
LSC	Lens Shading Calibration(LSC) data needed to format LSC data.
LSCData	isAvailable – Indicates that LSC data is available or not in the OTP data buffer and valid values are true/false. If this is false, all the remaining fields under LSC node will be ignored. **Example**: true endianness – Type of the endianness. valid values are BIG or LITTLE Example: LITTLE
lightInfo	Contains the type of the illuminant and corresponding LSC gain values.
illuminantType	Type of the illuminant for which LSC data is available. Supported illuminants are D65, TL84, A, D50, H.
rGainMSB	offset – Position of the MSB of the rGain value in the buffer. mask – Represents the number of valid bits of the data.
rGainLSB	offset – Position of the LSB of the rGain value in the buffer. mask – Represents the number of valid bits of the data.
grGainMSB	offset – Position of the MSB of the grGain value in the buffer. mask – Represents the number of valid bits of the data.
grGainLSB	offset – Position of the LSB of the grGain value in the buffer. mask – Represents the number of valid bits of the data.

Thundercomm Confidential

Field	Description
gbGainMSB	offset – Position of the MSB of the gbGain value in the buffer. mask – Represents the number of valid bits of the data.
gbGainLSB	offset – Position of the LSB of the gbGain value in the buffer. mask – Represents the number of valid bits of the data.
bGainMSB	offset – Position of the MSB of the bGain value in the buffer. mask – Represents the number of valid bits of the data.
bGainLSB	offset – Position of the LSB of the bGain value in the buffer. mask – Represents the number of valid bits of the data.
meshHWRollOffSize	Mesh hardware rolloff size.
rIncrement	Value to be incremented to find the next rGainMSB and rGainLSB value Example: 5 that is, if rGainMSB offset is 28, then the next rGainMSB offset will be 28+5 = 33. Same applies to other increments and if rGainLSB offset is 24, then the next rGainMSB offset will be 24+5 = 29. Same applies to other increments.
grIncrement	Value to be incremented to find next grGainMSB and grGainLSB values.
gbIncrement	Value to be incremented to find next gbGainMSB and gbGainLSB values.
bIncrement	Value to be incremented to find next bGainMSB and bGainLSB values.

5. Bring up actuator

5.1. Actuator software configuration

Actuator specific XML

Every actuator has an associated configuration XML file that is used to define features such as, but not limited to, power settings, code step, and initialization settings. Entire settings are enclosed in the < actuatorDriver></ actuatorDriver> node of this XML.

Actuator information node

This node contains the information related to the actuator driver configuration parameters.

Table 2-28: Actuator slave information

Field	Description
slaveInfo	Master node that stores slave information used to communicate with actuator.
actuatorName	Name of the actuator.
slaveAddress	Slave address used to talk to the actuator.
i2cFrequencyMode	I2C frequency mode of slave. **Example**: FAST

Thundercomm Confidential

Field	Description
	Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz) • CUSTOM (custom frequency in DTSI)
actuatorType	Actuator type Supported types are: • VCM • BIVCM
dataBitWidth	Data width in bits.
powerUpSequence	This node contains the power-up configuration sequence required to control the power to the device while closing it. **Example**: **<powerDownSequence> <powerSetting> <configType>MCLK</configType> <configValue>0</configValue> <delayMs>0</delayMs> </powerSetting> </ powerDownSequence>**
powerSetting	This node contains power configuration type, value, and delay.
configType	Power configuration type. **Example**: MCLK Supported types are: • MCLK • VANA • VDIG • VIO • VAF • RESET • STANDBY
configValue	Power configuration type. **Example**: 24000000
delayMs	Delay in milliseconds. **Example**: 1
powerDownSequence	This node contains the power-down configuration sequence required to control the power to the device while closing it **Example**: **<powerDownSequence> <powerSetting> <configType>RESET</configType> <configValue>0</configValue> <delayMs>0</delayMs> </powerSetting> </ powerDownSequence>**

Table 2-29: Actuator register information

Thundercomm Confidential

Field	Description
registerConfig	This node holds sequence of register configurations.
registerParam	Actuator register parameter configuration.
regAddrType	Register address type/ data size in bytes. **Example**: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type / data size in bytes. **Example**: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
registerAddr	Register address that is accessed.
registerData	Register data to be programmed.
operation	Actuator operation to be performed on the register. Supported values are: • WRITE_HW_DAMP • WRITE_DAC • WRITE • WRITE_DIR_REG • POLL • READ_WRITE
delayUs	Delay in micro seconds
hwMask	Hardware mask
hwShift	Number of bits to shift for the hardware
dataShift	Number of bits to shift for data

Table 2-30: Actuator init information

Field	Description
initSettings	Sequence of register settings to configure the device.
regSetting	Register setting configuration.
registerAddr	Register address that is accessed.
registerData	Data to be processed, read from, or written into the address.
regAddrType	Register address type/ data size in bytes. **Example**: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type / data size in bytes.

Thundercomm Confidential

Field	Description
	Example: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
Operation	Operation to be performed on the register. Supported values are: • READ • WRITE • POLL
delayUs	Delay in micro seconds.

Table 2-31: Actuator tuning information

Field	Description
tunedParams	Actuator tuning parameters.
initialCode	Initial DAC value.
regionParams	Actuator region parameters for all regions.
region	Actuator region parameters for single region.
macroStepBoundary	Macro step boundary in the near/forward direction.
infinityStepBoundary	Infinity step boundary in the far/backward direction.
codePerStep	Code per step.
qValue	qvalue to convert float/double to integer format.
forwardDamping	Actuator scenario ringing and damping information in forward/near direction.
backwardDamping	Actuator scenario ringing and damping information in backward/far direction.
ringingScenario	Actuator ringing scenario value.
scenarioDampingParams	Actuator damping parameters for all scenarios element for dampingStep.
scenario	Actuator damping parameters for all regions element for dampingStep.
region	Actuator damping parameters for a single region.
dampingStep	Actuator damping step.
dampingDelayUs	Actuator damping delay in micro seconds is applied after programming damping step.
hwParams	Actuator hardware parameters.

Table 2-32: Actuator regulator

Field	Description
compatible	Data type that this node corresponds to.
reg	Points to the node ID.
regulator-name	LDO regulator name.

Thundercomm Confidential

Table 2-33: Actuator regulator (cont.)

Field	Description
regulator-min-microvolt	Minimum voltage for this LDO.
regulator-max-microvolt	Maximum voltage for this LDO.
regulator-enable-ramp-delay	The time taken, in microseconds, for the supply rail to reach the target voltage, ±whatever tolerance the board design requires. This property describes the total system ramp time required due to the combination of internal ramping of the regulator itself and board design issues such as, trace capacitance and load on the supply.
enable-active-high	Enable with active high.
gpio	GPIO to be used with this node.
vin-supply	Input voltage supply node name.

5.2. Actuator hardware configuration

Kernel hardware configuration

Depending on the LED flash hardware, OEMs can decide which type of interface driver to configure. Some LED Flash hardware requires a power supply at input to turn it on or off. For such LED Flash hardware, OEMs can use a PMIC-based LED Flash driver to supply current or power from the PMIC IC. This driver is simple and just calls PMICAPIs to control the current or power level for different Flash states. Other LED Flash hardware is programmed with register settings to turn it on or off. For that hardware, OEMs can use either QUP or I2C-based LED Flash drivers. The node entry in the device tree file changes based on the type of LED Flash driver – PMIC-based, I2C-based, or CCI-based.

For more details and an explanation of each field in the device tree file, refer to the kernel at: *kernel/Documentation/devicetree/bindings/media/video$ vi msm-camera-flash.txt.*

Actuator node structure

Different actuator node structures are required for different interfaces. There is no support for SPI interface for actuator.

Table 2-34: SOC-based actuator driver

Field	Description
compatible	Data type that this node corresponds to.
reg	Should contain the I2C slave address of the actuator and length of data field, which is 0x0.
regulator-name	A string used as a descriptive name for regulator outputs.
regulator-min-microvolt	Smallest voltage to set.
regulator-max-microvolt	Largest voltage to set.
regulator-enable-ramp-delay	Ramp delay for regulator.
gpio	Contains phandle to GPIO controller node and array of #gpio-cells specifying a particular GPIO.

Thundercomm Confidential

Field	Description
vin-supply	phandle to a regulator that powers this actuator.

Example:

```
actuator_regulator: gpio-regulator@0 {
compatible = "regulator-fixed";
reg = <0x00 0x00>;
regulator-name = "actuator_regulator";
regulator-min-microvolt = <2800000>;
regulator-max-microvolt = <2800000>;
regulator-enable-ramp-delay = <100>;
enable-active-high;
gpio = <&tlmm 27 0>;
vin-supply = <&pmi8998_bob>;
};
```

Table 2-35: CCI-based actuator driver

Field	Description
cell-index	Contains a unique identifier to differentiate between multiple actuators.
reg	Corresponds to the cell-index of the node.
compatible	Data type to which this node corresponds to.
qcom,cci-master	CCI node that drives this node.
cam_vaf-supply	Contains the regulator from which AF voltage is supplied.
qcom,cam-vreg-name	Contains the names of all regulators needed by this actuator.
qcom,cam-vreg-min-voltage	Contains the minimum voltage level in microvolts for regulators mentioned in qcom, cam-vreg-name property (in the same order).
qcom,cam-vreg-max-voltage	Contains the maximum voltage level in microvolts for regulators mentioned in qcom, cam-vreg-name property (in the same order).
qcom,cam-vreg-op-mode	Contains the maximum current in µA that is required from the regulators mentioned in the qcom, cam-vreg-name property (in the same order).

Example:

```
actuator_front: qcom,actuator@1 {
cell-index = <1>;
reg = <0x1>;
compatible = "qcom,actuator";
qcom,cci-master = <1>;
cam_vaf-supply = <&actuator_regulator>;
qcom,cam-vreg-name = "cam_vaf";
qcom,cam-vreg-min-voltage = <2800000>;
qcom,cam-vreg-max-voltage = <2800000>;
qcom,cam-vreg-op-mode = <0>;
};
```

6. Flash software configuration

Flash-specific XML

Flash has an associated configuration XML file used to define features such as, but not limited to the type of flash, max current, max duration, and flash related information. Entire settings are enclosed in the < flashDriverData ></ flashDriverData > node of this XML.

Flash information node

Table 2-36: Flash driver data

Field	Description
flashDriverData	This node has all the information regarding the flash type and its parameters.
flashName	Name of the flash.
flashDriverType	Flash driver type. Supported values are: • PMIC • I2C
powerUpSequence	Sequence to power up the flash.
powerDownSequence	Sequence to power down the flash.
i2cInfo	Settings for an I2C-based flash.
flashInformation	Flash-related trigger information.

Table 2-37: Flash I2C information

Field	Description
slaveAddress	8-bit or 10-bit I2C slave write address.
regAddrType	Register address type/data size in bytes. **Example**: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type or data size in bytes. **Example**: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
i2cFrequencyMode	I2C Frequency mode of slave. **Example**: FAST Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz)

Thundercomm Confidential

Field	Description
	• CUSTOM (custom frequency in DTSI)
flashInitSettings	Sequence of I2C register settings to initialize flash.
flashOffSettings	Sequence of I2C Register settings to turn off flash.
flashLowSettings	Sequence of I2C Register settings to turn on low flash.
flashHighSettings	Sequence of I2C Register settings to turn on high flash.
regSetting	This node holds one register configuration and forms a unit of large flash-register-settings sequence.
registerAddr	Register address that is accessed.
registerData	If the operation is WRITE, registerData is the data value to be written into the specified register address. If the operation is READ, registerData is the number of bytes to be read from the specified register address.
regAddrType	Type of register address.
regDataType	Type of register data.
operation	Type of operation Supported values: • WRITE • READ • POLL
delayUs	Delay in microseconds. If not explicitly specified, the delay is zero.

Table 2-38: Flash trigger information

Field	Description
triggerInfo	Flash trigger information.
maxFlashCurrent	Maximum flash current of the trigger in mA.
maxTorchCurrent	Maximum Torch current of the trigger in mA.
maxFlashDuration	Maximum flash duration of the trigger in ms.

Flash software configuration

Kernel hardware configuration

Depending on the LED flash hardware, OEMs can decide which type of interface driver to configure. Some LED flash hardware requires a power supply at input to turn it on or off. For such LED flash hardware, OEMs use a PMIC-based LED flash driver to supply current or power from the PMIC IC. This driver is simple and just calls PMIC APIs to control the current and power level for different flash states. Other LED flash hardware is programmed with register settings to turn it on/off. For that hardware, OEMs can use either QUP or I2C-based LED flash drivers. The node entry in the device tree file changes based on the type of LED Flash driver – PMIC-based, I2C-based, or CCI-based.

The kernel at *kernel/Documentation/devicetree/bindings/media/video$ vi msm-camera-flash.txt* provides more details and explanation of each field in the device tree file.

Flash node structure

Thundercomm Confidential

Different flash structures are required for different interfaces. There is no support for SPI interface for flash.

Table 2-39: PMIC-based LED flash driver

Field	Description
cell-index	Specifies the node ID.
reg	Increments based on the cell index.
compatible	Data type that this node corresponds to.
flash-source	Should contain the array of phandles to flash source nodes.
torch-source	Should contain the phandle to Torch source node.
switch-source	Should contain the phandle to switch source node. Trigger dual led at same time to avoid sync issues.
status	Whether this node is enabled or disabled.

Example:

```
&soc {
led_flash_rear: qcom,camera-flash@0 {
cell-index = <0>;
reg = <0x00 0x00>;
compatible = "qcom,camera-flash";
flash-source = <&pmi8998_flash0 &pmi8998_flash1>;
torch-source = <&pmi8998_torch0 &pmi8998_torch1>;
switch-source = <&pmi8998_switch0>;
status = "ok";
};
};
```

Table 2-40: QUP/I2C-based LED flash driver

Field	Description
cell-index	Specifies the node ID.
reg	Increments based on the cell index.
qcom,slave-id	Slave address for the I2C communication with flash hardware.
compatible	Data type to which the node corresponds to.
qcom,flash-name	Flash name.
qcom,flash-type	Flash type. Supported values: • LED flash • Strobe flash • Simple LED flash controlled by one GPIO
qcom,gpio-no-mux	Specifies the GPIO mux type. • 1 = GPIO mux is not available • 0 = Otherwise
gpios	Contains phandle to GPIO controller node and array of #gpio-cells specifying a GPIO (controller-specific).
qcom,gpio-flash-en	Contains the index to GPIO used by flash enable pin of flash.

Thundercomm Confidential
Sent: 2021-12-31 9:16:31 AM CST
Sent by: ycammoonvision.com

Field	Description
qcom,gpio-flash-now	Contains the index to GPIO used by flash's 'flash now' pin.
qcom,gpio-flash-reset	Contains the index to GPIO used by flash reset pin of flash.
qcom,gpio-req-tbl-num	Contains the index to GPIO-specific to this flash.
qcom,gpio-req-tbl-flags	Contains direction of GPIO present in qcom, gpio-req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Contains name of GPIOs present in qcom, gpio-req-tbl-num property (in the same order).

Example:

```
&i2c {
led_flash0: qcom,led-flash@60 {
cell-index = <0>;
reg = <0x60>;
qcom,slave-id = <0x60 0x00 0x0011>;
compatible = "qcom,led-flash";
qcom,flash-name = "adp1600";
qcom,flash-type = <1>;
qcom,gpio-no-mux = <0>;
gpios = <&msmgpio 18 0>,
<&msmgpio 19 0>;
qcom,gpio-flash-en = <0>;
qcom,gpio-flash-now = <1>;
qcom,gpio-req-tbl-num = <0 1>;
qcom,gpio-req-tbl-flags = <0 0>;
qcom,gpio-req-tbl-label = "FLASH_EN",
"FLASH_NOW";
};
};
```

Table 2-41: CCI-based LED flash driver

Field	Description
cell-index	Specifies the node ID.
reg	Increments based on the cell index.
qcom,slave-id	Slave address for the I2C communication with flash hardware.
compatible	Data type to which this node corresponds to.
label	Contain unique flash name to differentiate from other flash.
qcom,flash-type	Flash type Supported values: • LED flash • Strobe flash • Simple LED flash, controlled by one GPIO
pinctrl-names	Defined if a target uses pinctrl framework.
pinctrl-0	Pinctrl handle for pin0.
pinctrl-1	Pinctrl handle for pin1.

Field	Description
qcom,gpio-no-mux	Specifies the GPIO mux type. • 1 = GPIO mux is not available • 0 = Otherwise
gpios	Contain phandle to GPIO controller node and array of #gpio-cells specifying the GPIO (controller specific).
qcom,gpio-flash-en	Contains an index to GPIO used by flash enable pin of flash.
qcom,gpio-flash-now	Contains an index to GPIO used by flash now pin of flash.
qcom,gpio-flash-reset	Contains an index to GPIO used by flash reset pin of flash.
qcom,gpio-req-tbl-num	Contains an index to GPIOs specific to this flash.
qcom,gpio-req-tbl-flags	Contains the direction of GPIOs present in qcom, gpio-req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Contains the name of GPIOs present in qcom, gpio-req-tbl-num property (in the same order).
qcom,cci-master	Contain I2C master ID to be used for this flash. Supported values: • 0 – MASTER_0 • 1 – MASTER_1

Example:

```
$cci {
led_flash0: qcom,led-flash@66 {
cell-index = <0>;
reg = <0x66>;
qcom,slave-id = <0x66 0x00 0x0011>;
compatible = "rohm-flash,bd7710";
label = "bd7710";
qcom,flash-type = <1>;
qcom,gpio-no-mux = <0>;
qcom,enable_pinctrl;
pinctrl-names = "cam_flash_default", "cam_flash_suspend";
pinctrl-0 = <&cam_sensor_flash_default>;
pinctrl-1 = <&cam_sensor_flash_sleep>;
gpios = <&msm_gpio 36 0>,
<&msm_gpio 32 0>,
<&msm_gpio 31 0>;
qcom,gpio-flash-reset = <0>;
qcom,gpio-flash-en = <1>;
qcom,gpio-flash-now = <2>;
qcom,gpio-req-tbl-num = <0 1 2>;
qcom,gpio-req-tbl-flags = <0 0 0>;
qcom,gpio-req-tbl-label = "FLASH_RST",
"FLASH_EN",
"FLASH_NOW";
qcom,cci-master = <0>;
};
}
```

Confidential
Thundercomm 2025-01-31 9:16:31 AM CST
Stefan@ucam.moonvision.com

Appendix 1. Intellectual Property Statement

This document itself and the proprietary technical information contained herein is the property of Thundercomm. Without prior written authority of Thundercomm, no one shall copy, modify, or reprint the whole or part of this document. Offenders are liable for all related damages, loss, and litigations.

Documentation content is subject to change without notice. Thundercomm Technology makes constant improvements on the documentation.

Thundercomm Confidential
2025-01-31 9:16:31 AM CST
Stefan_ucam.moorvision.com

Appendix 2. Trademarks

Hexagon is a trademark of Qualcomm Technologies, Inc. and/or its affiliated companies. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All other trademarks are the property of their respective owners.

Thundercomm Confidential
2025-01-31 9:16:31 AM CST
Stefan_ucam.moorvision.com